

P2P Engine

P2P Engine is a project-memory system for work in which people and AI agents must share context without losing the reasoning, constraints and ownership behind important choices. It keeps that memory local, inspectable and versioned, then derives smaller views that help each participant recover the part of the project relevant to the task at hand.

Purpose and intended outcome

The central objective is to preserve project intent as durable, auditable knowledge. Rough ideas should be able to develop into a coherent project without forcing a person or an agent to reread every historical artifact before taking the next step.

The engine is designed around local files and Git history. This keeps project knowledge portable and reviewable while allowing deterministic tools to validate, summarize and retrieve it. The intended result is not an autonomous project manager: it is a reliable memory and mediation layer that helps the accountable people make informed decisions and gives agents bounded context for useful work.

People, agents and authority

The primary users are project owners, solo developers, small technical teams, maintainers and the coding or planning agents that assist them. The owner remains responsible for decisions that establish or end project authority. Agents may analyze evidence, propose alternatives, prepare candidates and carry out explicitly permitted operations, but they do not silently decide on the owner's behalf.

This distinction is reflected in the product surfaces. Read-only analysis can remain lightweight. Persistent changes use declared commands, permission checks and, where the impact requires it, a preview followed by explicit confirmation. Generated instructions adapt these boundaries to supported agent environments while retaining a shared baseline.

From intent to actionable project memory

A typical flow begins with an idea that is explored and compared with existing project knowledge. The resulting proposal records the problem, intended outcome, alternatives, risks and acceptance conditions. Choices and conflicts can be made explicit instead of being buried in conversation.

When a direction is selected, operational change descriptions, work metadata and software specifications can be derived for delivery. These artifacts bridge design and

execution, but they do not prove that external implementation occurred. If a prior direction later becomes unsuitable, its history remains available and its authority is revised through explicit events rather than by erasing the earlier record.

How project memory is organized

Human-authored Markdown and structured YAML hold the durable project record. Around that record, the engine builds registries, a vertical-aware project view, decision context, progress views and ordered next actions. These derivatives are rebuildable and are not allowed to become competing sources of truth.

Retrieval is intended to be bounded and explainable. A request should receive nearby decisions, constraints, relationships and evidence instead of a generic prefix of the proposal archive. Provenance, authority, completeness and freshness remain attached to the result so that compression does not hide uncertainty or turn a heuristic association into a decision.

Interfaces and integrations

The command-line interface is the reference local surface for operating a workspace. A local MCP server exposes domain-specific tools to compatible agents and keeps privileged or unavailable operations explicit rather than emulating them silently. Generated adapter instructions let several agent clients coexist in the same repository.

Git provides durable history and optional managed collaboration. Project-local Python environments and versioned release artifacts provide the runtime. Downstream specification tools and publication renderers consume derived outputs. External provider actions remain capability-limited and permission-gated.

Vertical definition and project completeness

Each project selects a vertical that describes the essential questions and sections for its domain. For a software project, that lens covers purpose, users, scope, workflows, data, integrations, quality constraints, validation, risks and important decisions. The vertical guides completeness; it is not a rigid table of contents for every output.

Definition evidence and proposal evidence remain distinct. The engine can suggest likely relationships, but only declared evidence contributes authority. Gaps become structured questions for the owner, and answers become candidate definition changes that must be previewed and confirmed before they alter the project record.

Specifications, handoffs and publication

Software specifications are generated from governed project direction and linked change scope. They are implementation handoffs: downstream teams or tools may use them to build software, while P2P Engine remains concerned with the quality and traceability of the project knowledge rather than claiming that the implementation was completed.

The visible project export provides complete research material. Human publication adds an editorial model that reorganizes the evidence for a reader who does not know the upstream workflow. Curation, validation, PDF rendering and owner review are separate stages so that a polished file cannot imply approval by itself.

Runtime, schema and safe change

Runtime compatibility and workspace-schema compatibility are independent. A project declares the compatible engine range and recommended runtime, while the workspace declares its data-layout version. The supported Python baseline is 3.11 or newer.

Schema transitions are explicit and forward-moving. Migration planning is read-only; application checks the reviewed plan and current preimages, preserves unknown material by default and exposes recovery if an interruption occurs. Multi-file changes are expected to validate complete candidates and commit atomically. Read-only status and context operations should remain deterministic and free of persistent writes.

Boundaries, assumptions and risks

The local engine includes project initialization, governed memory, decision support, vertical definition, retrieval, validation, agent integration, specifications and exports. Hosted multi-tenant operation, autonomous owner decisions, unrestricted cloud or Git-provider mutations, arbitrary edits to governed state and guaranteed public-registry distribution are outside this product boundary.

The design assumes writable local storage, Git history, a compatible project-local runtime and an accountable owner available for semantic decisions. Adoption in a different operating environment must validate those assumptions.

The principal risks are stale derived views, partial writes, concurrent source drift, automatic migration that changes meaning, heuristic evidence being mistaken for authority and generated publications implying review that never occurred. The design counters them with fingerprints, freshness checks, bounded provenance, explicit authority, atomic transactions, preserve-by-default migration and separate editorial and owner gates. These controls reduce risk; they do not justify hiding residual uncertainty.

What success looks like

A successful workspace can be initialized, inspected and migrated through supported interfaces. People can move from intent to a reviewable project definition, retrieve relevant context, prepare downstream specifications and validate the resulting state without manual repair of governed files.

The system should make uncertainty visible, keep owner-controlled actions explicit, preserve historical reasoning and allow every derived view to be rebuilt from durable evidence. Compatibility checks and focused plus full test suites provide implementation-level evidence that these contracts continue to hold.

Contributions

The current summary is based on 204 explicitly recorded contribution records.

Contributor	Recorded share
local	44.12%
codex	43.63%
davide-via-codex	2.94%
owner	2.94%
owner-via-codex	2.94%
bootstrap	2.45%
intake:INTAKE-001	0.49%
intake:INTAKE-002	0.49%

Percentages are shares of explicitly recorded contribution records; they do not measure effort, quality, merit, ownership, code authorship, or intellectual property.